Flask Development → Raspberry Pi Production Workflow

This document outlines a clean, reproducible workflow for developing a Flask application on a local machine while deploying and running it in production on a Raspberry Pi. The workflow uses GitHub as the synchronization layer and environment variables to ensure the Pi always runs production settings.

---

1. Overview

This workflow ensures:

- Local development always uses **DevConfig**
- Raspberry Pi always uses **ProdConfig**
- GitHub stores all code, including both config classes
- Deployment is as simple as `git pull` on the Pi
- No risk of accidentally running development settings in production

Machine	Environment Variable	Config Loaded
Dev Computer	FLASK_ENV=development	DevConfig
Raspberry Pi	FLASK_ENV=production	ProdConfig

LEGEND:

code	Python code
commands	Commands are run either in a Command prompt or Shell

3. Configuration File (`config.py`)

Define separate configuration classes for development and production.

```
class BaseConfig:
    <settings to share across DEV and PROD environments>

class DevConfig(BaseConfig):
    DEBUG = True
    ENV = "development"
    DATABASE_URI = "sqlite:///dev.db"

class ProdConfig(BaseConfig):
    DEBUG = False
    ENV = "production"
    DATABASE_URI = "sqlite:///prod.db"
```

---

4. Application Factory (`app.py`)

Load configuration based on an environment variable.

```
env = os.getenv("APP_ENV", "development")
config = DevConfig if env == "development" else ProdConfig
```

---

5. Local Development Setup

On your development machine, set:

CMD: `set FLASK_ENV=development`

Check: `echo %FLASK_ENV%`

Run the app: `python run.py`

Commit code normally:

```
git add .
git commit -m "Sprint update"
git push
```

---

6. Raspberry Pi Production Setup

Set the environment variable once:

echo "FLASK_ENV=production" | sudo tee -a /etc/environment
source /etc/environment

echo "FLASK_ENV=production" | sudo tee -a /etc/environment	adds variable=value into '/etc/environment' Then appends () the variable=value to the file with root permissions.
source /etc/environment	loads environment variables from a file **into your current shell session**
cat /etc/environment echo $FLASK_ENV	QA that your variable exists

echo $FLASK_ENV

If using a systemd service, include: (SEE BELOW)

Environment="FLASK_ENV=production"

The Pi will always load `ProdConfig`.

Linux "two worlds" problem: SSH sessions and VNC sessions do NOT share the same environment. They behave like completely different login shells, so environment variables set in one will not automatically appear in the other.

chmod +x run_on_pi.sh #changed permissions and created a new run file
chmod +x run.py #changed permissions

---

7. Deployment Workflow

On the Raspberry Pi:

```
cd /path/to/app
git pull
sudo systemctl restart flaskapp
```

This updates the code and restarts the production service.

---

8. Optional: Secure Production Secrets

Create a file:

/etc/myapp.env

Add secrets:

```
SECRET_KEY=super-secret
DATABASE_URI=postgres://...
```

Load via systemd:

```
EnvironmentFile=/etc/myapp.env
```

---

9. Optional: Flask App at StartUp

What *systemd* actually is:

> Systemd is the **init system** and **service manager** used by most modern Linux distributions

It is the thing that:

- boots the system
- starts background services
- restarts crashed processes
- manages logs

- controls daemons (long-running programs)

🧠 If you add your program to the service, it

- starts automatically on boot
- restarts if it crashes
- runs in the background
- has its own logs
- has its own environment variables
- doesn't require you to stay logged in

This is exactly how you'll want to run your Flask app on the Raspberry Pi.

🧩 Example: Running your Flask app with systemd

You create a service file:

/etc/systemd/system/groceries.service

Inside:

```
[Unit] Description=Groceries Flask App
After=network.target
[Service]
User=pi
WorkingDirectory=/home/pi/Desktop/GroceriesApp
Environment=APP_ENV=production
ExecStart=/usr/bin/python3 app.py
Restart=always
[Install]
WantedBy=multi-user.target
```

BASH:

```
sudo systemctl daemon-reload
sudo systemctl enable groceries.service
sudo systemctl start groceries.service
```

logs to `journalctl`

- sudo journalctl -u groceries.service -f
- sudo journalctl -u groceries.service

🔍 How to inspect systemd services

- See all services:

    - systemctl list-units --type=service
- Check your service status:
    - sudo systemctl status groceries.service
- View logs:
    - sudo journalctl -u groceries.service -f

If you want, I can help you write a perfect `groceries.service` file tailored to your project structure and environment setup.